Precision Unified Automation Framework

Architecture Documentation

Prepared by: QAlasers

Role: QA Automation Engineer

Date: April 2026

Table of Contents

1 Precision Unified Automation Framework - Architecture Documentation 4
1.1 Overview 4
1.2 Architecture Principles 4
1.2.1 1. Separation of Concerns 4
1.2.2 2. Modular Design 4
1.2.3 3. Design Patterns 4
1.3 High-Level Architecture 4
1.4 Module Architecture 5
1.4.1 APIAutomation Module 5
1.4.2 UIAutomation Module 7
1.4.3 Common Module 9
1.5 Data Flow Architecture 10
1.5.1 API Test Data Flow 10
1.5.2 UI Test Data Flow 11
1.6 Configuration Management 12
1.6.1 Centralized Configuration Strategy 12
1.6.2 Configuration Files Structure 12
1.7 Parallel Execution Architecture 13
1.7.1 Parallel Execution Strategies 13
1.7.2 Thread Safety Implementation 13
1.8 Reporting Architecture 13
1.8.1 Multi-Format Reporting Strategy 13
1.9 Error Handling Architecture 14
1.9.1 Exception Handling Strategy 14
1.9.2 Retry Mechanism Architecture 14
1.10 Security Architecture 15
1.10.1 Credential Management 15
1.11 Performance Optimization Architecture 15
1.11.1 Performance Strategies 15
1.12 Scalability Architecture 16
1.12.1 Scalability Design Principles 16

1.13 Technology Integration Architecture 17
1.13.1 Technology Stack Integration 17
1.14 Future Extension Architecture 18
1.14.1 Extensibility Design 18
1.15 Conclusion 18

1 Precision Unified Automation Framework - Architecture Documentation

1.1 Overview

The Precision Unified Automation Framework is a hybrid testing solution that combines UI and API automation capabilities within a structured Maven multi-module architecture. This document outlines the architectural design, component relationships, and implementation patterns used in the framework.

1.2 Architecture Principles

1.2.1 1. Separation of Concerns

- **UI Automation**: Isolated Selenium WebDriver logic
- **API Automation**: Dedicated RestAssured implementation
- **Common Utilities**: Shared components across modules
- **Configuration**: Centralized configuration management

1.2.2 2. Modular Design

- **Multi-module Maven structure** for maintainability
- **Independent test execution** for each module
- **Shared dependencies** through Common module
- **Scalable architecture** for future enhancements

1.2.3 3. Design Patterns

- **Page Object Model (POM)** for UI elements
- **Factory Pattern** for test data generation
- **Builder Pattern** for API request construction
- **Singleton Pattern** for configuration management

1.3 High-Level Architecture

```
Precision Unified Automation Framework
|
|-- APIAutomation Module
|   |-- Client Layer (ApiClient, RequestBuilder)
|   |-- Validation Layer (SchemaValidator, ResponseValidator)
|   |-- Test Data Layer (UserDataFactory, ProductDataFactory)
|   |-- Step Definitions (TC1-TC6)
|   `-- Test Runner (ApiTestRunner)
|
|-- UIAutomation Module
|   |-- Page Object Layer (HomePage, LoginPage, etc.)
```

```
|   |-- Utilities Layer (SeleniumUtils, WaitUtils)
|   |-- Step Definitions (TC1-TC5)
|   |-- Test Runner (TestRunner)
|   `-- Reporting Layer (ExtentTestManager)
|
|-- Common Module
|   |-- Configuration (ConfigReader, FrameworkConfig)
|   |-- Utilities (ExcelReader, DateUtils, FileUtils)
|   |-- Exceptions (FrameworkException, DataNotFoundException)
|   `-- Shared Resources (TestData, Templates)
|
`-- Parent Module
    |-- Dependency Management
    |-- Build Configuration
    `-- Test Execution Orchestration
```

1.4 Module Architecture

1.4.1 APIAutomation Module

1.4.1.1 Layered Design Implementation

```
APIAutomation/
|
|-- Client Layer (com.framework.api.client)
|   |-- ApiClient.java
|   |   |-- Centralized HTTP client management
|   |   |-- Request/Response handling
|   |   `-- Connection pooling
|   |
|   |-- RequestBuilder.java
|   |   |-- HTTP request construction
|   |   |-- Header management
|   |   |-- Query parameter handling
|   |   `-- Request body building
|   |
|   `-- ResponseHandler.java
|       |-- Response parsing
|       |-- Error handling
|       `-- Status code validation
|
|-- Validation Layer (com.framework.api.utils)
|   |-- SchemaValidator.java
|   |   |-- JSON schema validation
|   |   |-- Response structure verification
|   |   `-- Schema file management
|   |
|   |-- ResponseValidator.java
|   |   |-- Response data validation
|   |   |-- Business rule verification
|   |   `-- Custom assertions
|   |
```

```
|   `-- TestDataValidator.java
|       |-- Input data validation
|       |-- Parameter verification
|       `-- Data integrity checks
|
|-- Test Data Layer (com.framework.api.factory)
|   |-- UserDataFactory.java
|   |   |-- User test data generation
|   |   |-- Random data creation
|   |   `-- Template-based data
|   |
|   |-- ProductDataFactory.java
|   |   |-- Product test data
|   |   |-- Search term generation
|   |   `-- Category data
|   |
|   `-- RequestDataFactory.java
|       |-- API request payloads
|       |-- Parameter combinations
|       `-- Edge case data
|
|-- Step Definitions (com.framework.api.stepdefinitions)
|   |-- TC1_GetAllProductsSteps.java
|   |-- TC2_GetSingleProductSteps.java
|   |-- TC3_CreateUserSteps.java
|   |-- TC4_DeleteUserSteps.java
|   |-- TC5_UpdateUserSteps.java
|   |-- TC6_NegativeValidationSteps.java
|   `-- CommonSteps.java
|
`-- Test Runner (com.framework.api.runner)
    `-- ApiTestRunner.java
        |-- Cucumber configuration
        |-- TestNG integration
        |-- Parallel execution setup
        `-- Report generation
```

1.4.1.2 API Layer Responsibilities

1. **Client Layer**
    - HTTP communication management
    - Request/response lifecycle
    - Connection and session management
    - Error handling and retry logic
2. **Validation Layer**
    - JSON schema validation
    - Response data verification
    - Business rule validation
    - Custom assertion implementation

3. **Test Data Layer**
    - Dynamic test data generation
    - Template-based data creation
    - Random data generation
    - Edge case data management

1.4.2 UIAutomation Module

1.4.2.1 Page Object Model Implementation

```
UIAutomation/
|
|-- Page Object Layer (PageObjects)
|   |-- HomePage.java
|   |   |-- Home page element locators
|   |   |-- Home page interactions
|   |   |-- Navigation methods
|   |   `-- Home page validations
|   |
|   |-- LoginPage.java
|   |   |-- Login form elements
|   |   |-- Authentication methods
|   |   |-- Error handling
|   |   `-- Login validation
|   |
|   |-- RegisterPage.java
|   |   |-- Registration form elements
|   |   |-- User input methods
|   |   |-- Form validation
|   |   `-- Account creation
|   |
|   |-- CartPage.java
|   |   |-- Shopping cart elements
|   |   |-- Cart operations
|   |   |-- Price calculations
|   |   `-- Cart validation
|   |
|   `-- ProductPage.java
|       |-- Product detail elements
|       |-- Product interactions
|       |-- Add to cart functionality
|       `-- Product validation
|
|-- Utilities Layer (utils)
|   |-- SeleniumUtils.java
|   |   |-- WebDriver management
|   |   |-- Element interaction helpers
|   |   |-- Browser operations
|   |   `-- Common Selenium methods
|   |
|   |-- WaitUtils.java
```

```
|   |   |-- Explicit wait strategies
|   |   |-- Fluent wait implementations
|   |   |-- Custom wait conditions
|   |   `-- Timeout management
|   |
|   |-- ScreenshotUtils.java
|   |   |-- Screenshot capture on failure
|   |   |-- Image file management
|   |   |-- Screenshot naming conventions
|   |   `-- Report integration
|   |
|   `-- ElementUtils.java
|       |-- Element visibility checks
|       |-- Element interaction validation
|       |-- Element state verification
|       `-- Custom element locators
|
|-- Step Definitions (stepdefinitions)
|   |-- TC1_RegisterSteps.java
|   |-- TC2_LoginSteps.java
|   |-- TC3_InvalidLoginSteps.java
|   |-- TC4_AddToCartSteps.java
|   |-- TC5_RemoveFromCartSteps.java
|   `-- CommonUISteps.java
|
|-- Test Runner (runner)
|   `-- TestRunner.java
|       |-- Cucumber configuration
|       |-- TestNG integration
|       |-- Browser setup/teardown
|       `-- Report generation
|
`-- Reporting Layer (reports)
    `-- ExtentTestManager.java
        |-- ExtentReports configuration
        |-- Test categorization
        |-- Screenshot integration
        `-- Report customization
```

1.4.2.2 UI Layer Responsibilities

1. **Page Object Layer**
    - Element locator management
    - Page interaction methods
    - Business logic encapsulation
    - Page state validation
2. **Utilities Layer**
    - WebDriver lifecycle management
    - Wait strategy implementation
    - Screenshot capture automation

    - Element interaction helpers
3. **Reporting Layer**
    - ExtentReports integration
    - Test categorization
    - Screenshot attachment
    - Custom report formatting

1.4.3 Common Module

1.4.3.1 Shared Components Architecture

```
Common/
|
|-- Configuration Layer (config)
|   |-- ConfigReader.java
|   |   |-- Properties file reading
|   |   |-- Environment variable handling
|   |   |-- Configuration validation
|   |   `-- Default value management
|   |
|   `-- FrameworkConfig.java
|       |-- Global framework settings
|       |-- Browser configuration
|       |-- API configuration
|       `-- Report configuration
|
|-- Utilities Layer (utils)
|   |-- ExcelReader.java
|   |   |-- Excel file operations
|   |   |-- Data reading/writing
|   |   |-- Sheet management
|   |   `-- Data validation
|   |
|   |-- DateUtils.java
|   |   |-- Date formatting
|   |   |-- Timestamp generation
|   |   |-- Date calculations
|   |   `-- Time zone handling
|   |
|   |-- FileUtils.java
|   |   |-- File operations
|   |   |-- Directory management
|   |   |-- Path operations
|   |   `-- File validation
|   |
|   |-- StringUtils.java
|   |   |-- String manipulation
|   |   |-- Text validation
|   |   |-- Data formatting
|   |   `-- Random string generation
|   |
```

```
|   `-- JsonUtils.java
|       |-- JSON parsing
|       |-- JSON validation
|       |-- Data extraction
|       `-- JSON transformation
|
|-- Exception Layer (exceptions)
|   |-- FrameworkException.java
|   |   |-- Base framework exception
|   |   |-- Error code management
|   |   |-- Error message formatting
|   |   `-- Exception chaining
|   |
|   `-- DataNotFoundException.java
|       |-- Data missing exceptions
|       |-- Configuration errors
|       |-- Resource not found
|       `-- Custom error handling
|
`-- Resources Layer (src/test/resources)
    |-- testdata/
    |   |-- User data files
    |   |-- Product data files
    |   |-- API response samples
    |   `-- Test configuration files
    |
    `-- templates/
        |-- Email templates
        |-- Report templates
        |-- Configuration templates
        `-- Data templates
```

1.5 Data Flow Architecture

1.5.1 API Test Data Flow

```
Test Execution
    |
    v
Step Definition
    |
    v
Test Data Factory
    |
    v
Request Builder
    |
    v
API Client
    |
    v
```

```
HTTP Request
   |
   v
Application API
   |
   v
HTTP Response
   |
   v
Response Handler
   |
   v
Schema Validator
   |
   v
Response Validator
   |
   v
Test Assertion
   |
   v
Report Generation
```

1.5.2 UI Test Data Flow

```
Test Execution
   |
   v
Step Definition
   |
   v
Page Object
   |
   v
Selenium Utils
   |
   v
WebDriver
   |
   v
Browser
   |
   v
Web Application
   |
   v
Page Response
   |
   v
Element Validation
   |
```

```
   v
Page Object
   |
   v
Test Assertion
   |
   v
Screenshot (on failure)
   |
   v
Report Generation
```

1.6 Configuration Management

1.6.1 Centralized Configuration Strategy

```
Configuration Sources (Priority Order)
1. Environment Variables
2. System Properties
3. Properties Files
4. Default Values
```

1.6.2 Configuration Files Structure

```
config/
|
|-- api.properties
|   |-- API endpoints
|   |-- Request timeouts
|   |-- Authentication settings
|   `-- Retry policies
|
|-- ui.properties
|   |-- Browser settings
|   |-- Wait strategies
|   |-- Screenshot settings
|   `-- Driver configurations
|
|-- framework.properties
|   |-- Global settings
|   |-- Report configurations
|   |-- Logging levels
|   `-- Parallel execution settings
|
`-- environment/
    |-- dev.properties
    |-- qa.properties
    |-- staging.properties
    `-- prod.properties
```

1.7 Parallel Execution Architecture

1.7.1 Parallel Execution Strategies

1. **Module Level Parallelism**
    - API and UI modules run independently
    - Separate JVM processes
    - Independent resource allocation
2. **Test Level Parallelism**
    - Multiple test classes execute concurrently
    - Thread-safe implementation
    - Resource isolation
3. **Scenario Level Parallelism**
    - Cucumber scenarios run in parallel
    - Data-driven parallel execution
    - Independent test data

1.7.2 Thread Safety Implementation

```
ThreadLocal Pattern
|
|-- WebDriver ThreadLocal
|   |-- Thread-safe browser instances
|   |-- Isolated browser sessions
|   `-- Concurrent test execution
|
|-- TestContext ThreadLocal
|   |-- Thread-safe test data
|   |-- Isolated configuration
|   `-- Independent report instances
|
`-- Report ThreadLocal
    |-- Thread-safe report generation
    |-- Isolated test results
    `-- Concurrent report writing
```

1.8 Reporting Architecture

1.8.1 Multi-Format Reporting Strategy

```
Test Execution
    |
    v
TestNG Reports
    |-- HTML Reports
    |-- XML Reports
    `-- JUnit Reports
    |
    v
Cucumber Reports
```

```
   |-- HTML Reports
   |-- JSON Reports
   `-- Pretty Reports
   |
   v
Allure Reports (API)
   |-- Interactive HTML
   |-- Test History
   |-- Categories
   `-- Attachments
   |
   v
ExtentReports (UI)
   |-- Interactive Dashboard
   |-- Screenshots
   |-- Test Categories
   `-- Timeline View
```

1.9 Error Handling Architecture

1.9.1 Exception Handling Strategy

```
Exception Hierarchy
|
|-- FrameworkException (Base)
|   |-- ConfigurationException
|   |-- DataException
|   |-- ValidationException
|   `-- ExecutionException
|
|-- UIException
|   |-- ElementNotFoundException
|   |-- ElementNotVisibleException
|   |-- BrowserException
|   `-- PageLoadException
|
|-- APIException
|   |-- RequestException
|   |-- ResponseException
|   |-- ValidationException
|   `-- SchemaException
|
`-- ReportingException
   |-- ReportGenerationException
   |-- ScreenshotException
   `-- AttachmentException
```

1.9.2 Retry Mechanism Architecture

```
Retry Strategy
|
|-- Test Level Retry
```

```
|   |-- Failed test retry
|   |-- Retry count configuration
|   `-- Retry interval management
|
|-- Request Level Retry (API)
|   |-- Network failure retry
|   |-- Timeout retry
|   `-- Server error retry
|
|-- Element Level Retry (UI)
|   |-- Element not found retry
|   |-- Stale element retry
|   `-- Element interaction retry
|
`-- Global Retry Configuration
    |-- Max retry attempts
    |-- Retry conditions
    `-- Retry backoff strategy
```

1.10 Security Architecture

1.10.1 Credential Management

```
Security Layers
|
|-- Environment Variables
|   |-- API credentials
|   |-- Database credentials
|   `-- Third-party service credentials
|
|-- Encrypted Properties
|   |-- Sensitive data encryption
|   |-- Key management
|   `-- Decryption at runtime
|
|-- Access Control
|   |-- Role-based access
|   |-- Permission management
|   `-- Audit logging
|
`-- Data Masking
    |-- Log data masking
    |-- Report data masking
    `-- Debug data masking
```

1.11 Performance Optimization Architecture

1.11.1 Performance Strategies

```
Performance Optimization
|
|-- Connection Pooling (API)
```

```
|   |-- HTTP connection reuse
|   |-- Keep-alive connections
|   `-- Connection timeout management
|
|-- Browser Optimization (UI)
|   |-- Browser session reuse
|   |-- Driver caching
|   `-- Resource loading optimization
|
|-- Test Data Optimization
|   |-- Data caching
|   |-- Lazy loading
|   `-- Memory management
|
|-- Report Optimization
|   |-- Report generation optimization
|   |-- Attachment compression
|   `-- Report cleanup
|
`-- JVM Optimization
    |-- Memory management
    |-- Garbage collection tuning
    `-- Thread optimization
```

1.12 Scalability Architecture

1.12.1 Scalability Design Principles

```
Scalability Considerations
|
|-- Horizontal Scaling
|   |-- Multiple test environments
|   |-- Distributed test execution
|   `-- Load balancing
|
|-- Vertical Scaling
|   |-- Resource optimization
|   |-- Memory management
|   `-- CPU utilization
|
|-- Test Data Scaling
|   |-- Dynamic data generation
|   |-- Data partitioning
|   `-- Data cleanup strategies
|
|-- Report Scaling
|   |-- Report partitioning
|   |-- Archival strategies
|   `-- Distributed reporting
|
`-- Configuration Scaling
```

```
   |-- Environment-specific configs
   |-- Feature flags
   `-- Dynamic configuration
```

1.13 Technology Integration Architecture

1.13.1 Technology Stack Integration

```
Technology Integration
|
|-- Selenium WebDriver
|   |-- Browser automation
|   |-- Element interaction
|   `-- Browser management
|
|-- RestAssured
|   |-- API testing
|   |-- HTTP client
|   `-- JSON validation
|
|-- TestNG
|   |-- Test execution
|   |-- Parallel execution
|   `-- Test configuration
|
|-- Cucumber
|   |-- BDD implementation
|   |-- Feature file parsing
|   `-- Step definition mapping
|
|-- Maven
|   |-- Build management
|   |-- Dependency management
|   `-- Lifecycle management
|
|-- ExtentReports
|   |-- HTML reporting
|   |-- Dashboard generation
|   `-- Screenshot integration
|
|-- Allure
|   |-- Advanced reporting
|   |-- Test history
|   `-- Interactive reports
|
`-- Log4j2
   |-- Logging framework
   |-- Log management
   `-- Log formatting
```

1.14 Future Extension Architecture

1.14.1 Extensibility Design

```
Extension Points
|
|-- New Test Types
|   |-- Mobile automation
|   |-- Desktop automation
|   `-- Performance testing
|
|-- New Reporting Formats
|   |-- PDF reports
|   |-- Excel reports
|   `-- Custom dashboards
|
|-- New Data Sources
|   |-- Database integration
|   |-- API data sources
|   `-- External data feeds
|
|-- New Environments
|   |-- Cloud environments
|   |-- Container-based testing
|   `-- Microservices testing
|
`-- New Integrations
    |-- CI/CD pipelines
    |-- Test management tools
    `-- Monitoring systems
```

1.15 Conclusion

The Precision Unified Automation Framework architecture is designed to be:

1. **Modular**: Clear separation of concerns and responsibilities
2. **Scalable**: Supports growth and extension
3. **Maintainable**: Easy to understand and modify
4. **Robust**: Comprehensive error handling and recovery
5. **Performant**: Optimized for speed and resource usage
6. **Secure**: Proper credential management and data protection

This architecture provides a solid foundation for enterprise-level automation testing while maintaining flexibility for future enhancements and modifications.

Document Version: 1.0.0
Last Updated: 2026-04-10

Author: QAlasers Team
Contact: tanyakonapala@gmail.com